

P3668R1

Defaulting Postfix Increment and Decrement Operations

Published Proposal, 2025-04-28

Authors:

[Matthew Taylor](#)

[Alex](#)

Audience:

SG17

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

Postfix increment and decrement operators have a near-universal canonical definition. We propose that this default should be codified into the language by being able to `= default` such operators.

Table of Contents

1 Revision History

1.1 Changes from R0:

2 Motivation and Scope

3 Proposal

3.1 Minor Edge Cases

3.1.1 Volatile Qualification

3.1.2 Return type of the prefix increment operator

3.1.3 Explicit Object Parameter Types

3.2 `= default` is a function body

4 Prior Art

4.1 Template Solutions via CRTP

4.2 Reflection Solutions

4.3 Free function template `operator++(T&, int)`

4.4 Automatically Generating More Operators

5 Alternatives Considered

5.1 Rewrite Rule

5.2 Supporting Alternative Semantics for Postfix Increment on Iterators

5.3 Defaulting Templates

6 Other Papers in this Space

6.1 P3662: Improve Increment and Decrement Operator Syntax

6.2 P2952: `auto& operator=(X&&) = default`

7 Effect on Existing Code

8 Implementation Experience

9 Proposed Wording

9.1 Wording relative to P2952

References

Informative References

§ 1. Revision History

§ 1.1. Changes from R0:

- Added discussion of what you can already do in defaulted functions ([§ 3.2 = default is a function body](#)) following BSI feedback.
- Added discussion of the potential to reduce boilerplate further via defaulting templates ([§ 5.3 Defaulting Templates](#)) following BSI feedback.
- Fix to typo in wording.

§ 2. Motivation and Scope

Postfix increment and decrement operators have a default behaviour which already exists in the mind of every C++ developer - make a copy, increment/decrement the original, and return the copy. The canonical implementation of this default can always be expressed entirely in terms of other operations (prefix increment/decrement and copy-construction), in a manner which is entirely agnostic to any other members or subobjects of the class, e.g.

```
auto copy{*this};  
++*this;
```

```
return copy;
```

We propose that this sensible default be codified into the language.

Since C++11 the language has had a mechanism in `= default` for expressing that a function has its default semantics in order to save the user from repetitive boilerplate. Our proposal is that postfix increment and decrement operators should be defaultable.

Currently	With Proposal
<pre>class foo{ int member; public: constexpr foo& operator++(){ ++member; return *this; } constexpr foo operator++(int){ auto copy{*this}; this->operator++(); return copy; } };</pre>	<pre>class foo{ int member; public: constexpr foo& operator++(){ ++member; return *this; } constexpr foo operator++(int) = default; };</pre>

This reduces boilerplate code while still expressing the universally-known meaning of the function. The majority of classes written with postfix operations can now simply default them and get an operator which does the right thing. It reduces the possibility for user error when writing such operators - typical classes won't need their implementation of such functions checked and debugged; and atypical ones will have their explicitly-defined operators stand out by virtue of not being defaulted.

Standardising a "default" behaviour for postfix operations would also allow us to shrink the library specification, by expressing operations which are specified to be equivalent to the canonical implementation as being default. This paper does not propose this at this time, but the authors intend to bring a paper to propose this pending interest in the language feature and have identified 49 candidate operations which could be defaulted and remain semantically equivalent to their current behaviour.

NOTE: Most of the examples in this paper are written in terms of the postfix increment operator. The scope of this paper equally covers the postfix decrement operator, however we do not repeat every example for both operators in the interests of saving reader time.

§ 3. Proposal

We propose that the postfix increment operator be a defaultable operator. On an instance `c` of a class of type `C`, the default behaviour should be equivalent to

```
C tmp = C{c};
++c;
return tmp;
```

We also propose that the postfix decrement operator be a defaultable operator, with the defaulted behaviour being equivalent to

```
C tmp = C{c};
--c;
return tmp;
```

In both cases, if the defaulted function does not have return type `C`, or if `C` is not complete in the context in which the function is defined as defaulted, the program is ill-formed.

Note that in order to write a specification which applies to all permutations of the postfix operator, we cannot refer to `this`; as it is possible to define postfix `operator++` and `operator--` as free functions or functions with an explicit object parameter. For example, we are proposing that all of the below postfix operations should be defaultable:

```
struct S0 {
    int v;
    S0& operator++() { ++v; return *this; }
    S0 operator++(int) = default;
};

struct S1 {
    int v;
    S1& operator++() { ++v; return *this; }
    S1 operator++(this S1&, int) = default;
};

struct S2 {
    int v;
    S2& operator++() { ++v; return *this; }
};
S2 operator++(S2&, int) = default;
```

If `C` does not have a copy constructor and corresponding prefix operation which is accessible from the context of the body of the defaulted function, it is defined as deleted. Unlike the comparison operators, it is not required for a free function postfix operator to be a friend of the class on which it is operating. `operator++(int)` does not need to

access the subobjects of the class in order to use its copy constructor or prefix operator, so making it a `friend` seems an unnecessary requirement. This decision does, however, rule out some very arcane examples of defaulting the postfix operator, such as:

```
class foo{
    //Private
    foo(const foo&);
    foo& operator++();
};

//Defined as deleted
foo operator++(foo&, int) = default;
```

Note that our spelling of initialization syntax of the copy is very slightly different from how the library wording specifies postfix operations on existing iterators, e.g. [\[istream.iterator.ops\]](#), to account for the possibility of a user-defined class defining its copy constructor as `explicit`. As the default meaning of a postfix operator is widely known to invoke a copy, a user who explicitly opts into that default meaning should be considered to have consented to a copy being created.

§ 3.1. Minor Edge Cases

Many details of the behaviour of defaulted operators is already covered in [\[dcl.fct.def.default\]](#) and we not intend to change the existing behaviour. However there are a handful of small edge cases to consider in this design. Our baseline to determine how a defaulted postfix operator should behave is to consider the behaviour when `= default` is replaced with the three-line canonical definition. If it results in well-formed code with the correct semantics we consider it valid; and if it does not, we do not. However, some notable cases include:

§ 3.1.1. Volatile Qualification

`volatile` increment and decrement operations are deprecated as of [\[P1152\]](#). We do not intend to break with the status quo, so consider that defaulted postfix operations acting on a `volatile` object are permitted, but deprecated.

§ 3.1.2. Return type of the prefix increment operator

It is possible for the user to define a prefix increment operator for class `C` which has a different return type from the expected default of `C&`. It may be tempting when specifying a default postfix operation to require that the prefix operation must always return `C&` out of a desire to ensure that it does the right thing and prevent the generated default postfix operation from exhibiting surprising behaviour.

However, the return type of the prefix operator function alone is not enough to guarantee it does a "canonical" operation - it may return a surprising value (e.g. a reference to some other instance of `C`). The compiler cannot look ahead to the definition of a declared prefix operation, which may be in some other TU, at the point of injecting a

defaulted postfix definition; and even if we were to require that the full definition be visible, it could not in general prove that the returned value is the correct one. As such, it is impossible to specify that the prefix operator must always return a correct reference to the correct object. Additionally, the canonical implementation discards the return value of the prefix operator in all cases.

With this in mind, we do not impose requirements on the return type of the prefix operator.

§ 3.1.3. Explicit Object Parameter Types

There exists an occasionally-surprising feature of explicit object parameters, namely that the explicit object parameter is not required to be the type of the class on which the function is called. For example, the following code is well-formed:

```
struct S {  
    operator int() { return 42; }  
    auto operator++(this int, int) { /*...*/ }  
};
```

Which raises the question of whether such an operator should be defaultable. In such cases, a canonical function definition for postfix increment would still generate well-formed code. However, this does not follow the canonical semantics of a default postfix operation, so should not be valid behaviour when specifying that a class has the default semantics.

Fortunately, this possibility is already accounted for in the current standard. Wording in [\[dcl.fct.def.default\]](#) requires that explicit object parameters of defaulted special member functions have type "reference to `C`". While postfix increment and decrement operators are not special member functions; we see no reason to break with the existing intent and allow other types of explicit object parameters there. As such, if the explicit object parameter is not of a reference type to `C`, the program is ill-formed.

§ 3.2. `= default` is a function body

It is worth noting that `= default` is just a function body which the implementation uses to inject the "default" semantics. It does not constrain the grammar of the function declaration. Defaulted functions are just as able to be marked as `noexcept`, to be constrained by `requires` clauses, or to feature other declaration-level constructs as their non-defaulted counterparts. We do not intend to change this, but we mention it to highlight that we do not need any further grammar changes to permit code such as the following:

```
template<typename Range>  
class my_generic_iterator{  
    //...
```

```

my_generic_iterator operator++(int) requires std::ranges::forward_range<Range> =
my_generic_iterator operator++(int){ /*...*/ }
};

```

Equally, the mechanism to = **default** postfix operations has no magic associated which might itself affect overload resolution. If the user does not declare a postfix operation for their class, no part of this proposal will generate rewritten candidates to make postfix operations valid. And, while it is not necessary, if the user wishes to be explicit that their class does not support postfix operations, they still have the option to explicitly delete the function.

§ 4. Prior Art

There are several existing approaches to attempt a similar reduction in boilerplate. Our position is that none of them quite find the best way to express user intent in code, and will examine them here.

§ 4.1. Template Solutions via CRTP

It is possible in current C++ to define a mixin class to attach the canonical postfix operation to derived classes, such as

```

struct incrementable{
    constexpr auto operator++(this auto& self, int){
        auto cpy{self};
        ++self;
        return cpy;
    }
};

class my_class : public incrementable{
    //...
    auto& operator++(){/*...*/}

    using incrementable::operator++;
};

```

Where `my_class` will inherit a postfix increment operation from `incrementable` which follows the canonical definition. This is a good solution, and indeed something similar to this is proposed in [P2727], but we find that it is still suboptimal. Every parent class which inherits from `incrementable` must explicitly make the postfix operator visible with a **using** expression, as the prefix operator in `my_class` shadows the postfix in `incrementable`. Mixins open the door to potentially confusing semantics, such as `my_class` being pointer-interconvertible with `incrementable`; and ultimately the user must still intrude into their class to place a declaration which might result in the canonical definition being applied but is counter-intuitively spelled as a **using** declaration.

§ 4.2. Reflection Solutions

[P3294] notes that generative reflection, when it arrives, could also be used to append postfix operations to classes by defining a metafunction to do the canonical thing, such that a user might:

```
struct C {  
    int i;  
  
    auto operator++() -> C& {  
        ++i;  
        return *this;  
    }  
  
    consteval { postfix_increment(); }  
};
```

This solution is analogous to the previous example - in both cases the user must define some other code elsewhere to manually implement the canonical operations and place a declaration in their class to attach those operations to it. As before, is a less direct way of spelling what this paper proposes - what the user really wants to say is "give me the default postfix operation"; and C++ already has a standard spelling for this with `= default`. We argue that such reflection-based solutions are well suited for specific problems where the user wishes to ensure that generated code can be programmatically tailored to the particular nature of the problem at hand or family of classes into which the definitions are intended to be injected. However, we seek to provide a general solution based on the universal default semantics of postfix operations. Doing this does not prevent the user from using reflection to generate more fine-grained solutions when that is the correct tool for the job, but the possibility of generative reflection does not prevent us from recognising the canonical default and providing a general solution.

There is also benefit in standardising the way to retrieve this default - multiple libraries might all ship their own `incrementable` or `postfix_increment()` variants, which may make slightly different design choices. For example, a library may choose to support increment, decrement, or both; or attempt to deduce properties of the class to which its operations are applied change behaviour accordingly. This runs the risk of cluttering code which uses them with several similar-sounding names which correspond to subtly different semantics; whereas `C operator++(int) = default` need not introduce additional names and has a single, clear meaning.

§ 4.3. Free function template `operator++(T&, int)`

It would be possible to define a free function template postfix operator, which automatically generates the correct operator overloads for the class, for example:

```
template<typename T>  
requires std::copyable<T> && requires(T t){  
    {++t} -> std::same_as<T&>;  
}  
  
constexpr T operator++(T& val, int){
```



```
T copy = T{val};
++val;
return copy;
}
```

This approach is opt-out, rather than opt-in, with every copyable and prefix-incrementable class automatically gaining postfix operations whether they make sense or not; and so opens the door to additional surprise features. Even if we wish to try to limit this through constraints, we cannot constrain for all possibilities. It resembles `std::rel_ops` which never managed to be the right way to add comparison operators to an existing class and have since been deprecated.

It would be possible to engineer an opt-in solution with template metaprogramming, such as some template boolean `enable_default_postfix_operations` which the user must specialise for every class that they define. We are generally not persuaded by this solution - it has the same issue as the previous alternatives in that the user must spell what they want in an exotic way, rather than in the way provided by the language. Similarly, the potential for a class' supported basic arithmetic operators being hidden in a standard library header could be a very easy source of confusion.

§ 4.4. Automatically Generating More Operators

We are not the first to propose the ability to synthesise postfix operations from prefix operators. This was previously suggested as part of [P1046], which sought to allow the majority of C++ arithmetic operators to be automatically generated from rewrites of other operators. The author has since parked that paper in favour of a more fine-grained proposal in [P3039], which is exclusively concerned with generation of `operator->()` and `operator->*`. We do not interpret this as a sign that there is no interest in automatic generation of postfix operations from their prefix counterparts, as P1046 sought to generate `x++` implicitly, and EWG feedback was consensus in favor of explicit syntax [P1046-EWG].

§ 5. Alternatives Considered

§ 5.1. Rewrite Rule

One alternative we considered for this change would be a rewrite rule, similar to the C++20 equality and comparison operator changes, such that `a++` could generate a rewritten candidate equivalent to `[&a]{auto copy{a}; ++a; return copy;}()`. This is a subset of what was proposed in [P1046]. The benefit here was a simplification in concept - a user who wished their class to be "incrementable" need only define `operator++()` to perform the simple increment operation and they would automatically get both operators which do the right thing. The issue with a rewrite rule is that increment and decrement operators are not as suitable for implicit operations as equality and comparison ones. The validity of `++a` does not strictly imply validity for `a++`, in the same way that `==` implies that `!=` is valid. Similarly, opt-out semantics make ill-formed code written today retroactively well-formed tomorrow, unless the author explicitly updates their library to delete functions which until now never needed to be declared.

Ultimately, a rewrite rule is difficult to specify and adds additional traps to the language, so the idea was dropped.

§ 5.2. Supporting Alternative Semantics for Postfix Increment on Iterators

Some generic iterators only use the canonical semantics when the underlying iterator operates on a forward range and falls back to an alternative implementation otherwise, which only increments the underlying iterator (e.g. on `std::move_iterator`). We do not propose supporting this semantic as part of the defaultable semantics for postfix increment, as it creates several pitfalls:

- It is asymmetric with `operator--`, as all ranges which support decrement are at least bidirectional.
- This is not the only alternative semantics for postfix increment, e.g. [the insert iterators](#) define it as a no-op which only exists to satisfy the requirements of *LegacyOutputIterator*.
- Not every class which supports postfix increment and decrement is an iterator; so a solution would need to do nothing surprising on classes which model arithmetic types and other usages.

It also increases rather than decreases the mental load on the developer to understand what a class does. For example, consider

```
class my_fun_iterator{
    //...
public:
    my_fun_iterator& operator++(){ ... }
    my_fun_iterator operator++(int) = default;
};

class my_input_iterator{
    //...
public:
    my_input_iterator& operator++(){ ... }
    void operator++(int) = default;
};
```

Do these two defaulted operations do the same thing? Intuitively, they should; but were we to support an "alternative" semantic we would break this intuition. The only information the user has to determine whether the defaulted operation would perform the "canonical" semantic or the "alternative" semantic is the return type of the function. Disambiguation by return type is not a path we want to walk down, as it is an entirely novel concept which has far broader implications than just getting the correct default behaviour for postfix increment.

While it is good to consider common alternatives which see some consistent use, this is not the universal canonical semantic of postfix increment and we do not seek to include it in the default definition.

§ 5.3. Defaulting Templates

There exists a hypothetical world where the boilerplate of defining the same postfix operator can be reduced even further. Consider this C++23 code:

```
class common_base {
    int m_Val{};
protected:
    int increment() { return ++m_Val; }
public:
    template<typename Self>
    Self& operator++(this Self& self) {
        self.increment();
        return self;
    }

    template<typename Self>
    Self operator++(this Self& self, int){
        auto copy{self};
        ++self;
        return copy;
    }

};
```

If `common_base` is the base of an extended hierarchy, then these templates will ensure that every derived class has the correct increment semantics (assuming that derived classes don't add additional relevant state or their own increment operator overloads). As `= default` is just a function body which is created to be equivalent to the function body of the postfix template in that example, it would be very tempting to allow such a function template to be defaulted. However, our current specification forbids postfix operator function templates from being defaulted, as this is consistent with the other defaultable functions currently in the language.

We believe that this is the sensible choice at present. While this reduction in boilerplate is desirable the implications of defaulting function templates are far broader than this example and have a different set of design questions to consider. We would prefer to see a general-purpose solution which might be extensible to other defaulted functions, and believe that such a solution should be its own paper.

§ 6. Other Papers in this Space

While we are not aware of any other papers seeking to automatically generate the canonical definition of postfix operators, there are some papers which are tangentially relevant because they are in a similar space. We shall examine how they may interact with this proposal.

§ 6.1. P3662: Improve Increment and Decrement Operator Syntax

[P3662] suggests an alternative syntax for prefix and postfix operations in order to more naturally disambiguate them, rather than use a phantom `int`:

```
struct S {  
    auto operator++ prefix() { ... } // ++s  
    auto operator++ postfix() { ... } // s++  
};
```

With new contextual keywords `prefix` and `postfix`; which are token-swapped directly onto the status-quo signatures to preserve ABI. We do not believe that there is any incompatibility between this paper and P3662. As `operator++ postfix()` maps directly onto a signature of `operator++(int)`, we anticipate that should P3662 be accepted that no further changes would be needed for the following to be valid code.

```
struct S {  
    auto operator++ prefix() { ... } // ++s  
    S operator++ postfix() = default // s++  
};
```

We have discussed this with the author of P3662, and he agrees that there is minimal possibility of compatibility issues between the two papers. While the design of P3662 seeks to evolve following committee feedback, we would be happy to collaborate to ensure that both papers are able to progress in parallel with minimal issues.

§ 6.2. P2952: `auto& operator=(X&&) = default`

[P2952] proposes that it should be valid to use placeholder return types in the signatures of explicitly defaulted functions, so long as those types would deduce to the correct return type for the function from the imaginary return statement in the function body. As we require that a defaulted postfix operation on class `C` have a return type of `C`; we specify that if the declared return type contains a placeholder type, it is deduced as if from an lvalue of type `C`. This leads to the following behaviour:

EXAMPLE 1

The behaviour of placeholder return types for a defaulted `operator++(int)`:

```
struct C{
    auto operator++(int) = default;           //Well-formed, deduces to C
    decltype(auto) operator++(int) = default; //Well-formed, deduces to C
    auto* operator++(int) = default;          //Ill-formed, deduction fails
    auto& operator++(int) = default;          //Ill-formed, C& is not C
    auto&& operator++(int) = default;         //Ill-formed, C& is not C
};
```

We anticipate no compatibility issues between the two papers. As P2952 is currently in Core for C++26, we include wording relative to it which can be used should it be accepted.

§ 7. Effect on Existing Code

We anticipate no effects on existing code. This change is strictly additive and so only opens up possibilities for new code.

§ 8. Implementation Experience

None yet.

§ 9. Proposed Wording

We tentatively propose making minimal alteration to [\[dcl.fct.def.default\]](#) and instead defining the behaviour and properties of defaulted postfix operations in a new clause [\[over.inc.default\]](#). This follows the lead of the C++20 comparison changes to defaulted functions, which were largely defined in their own [\[class.compare.default\]](#) clause. We will, however, modify it to note the existence of defaulted postfix operations:

Modify §9.6.2 [\[dcl.fct.def.default\]](#) as follows:

¹ A function definition whose *function-body* is of the form `= default ;` is called an *explicitly-defaulted definition*. A function that is explicitly defaulted shall

1.1 - be a special member function ([special]) ~~or a~~, comparison operator function ([over.binary], [class.compare.default]), or postfix increment or decrement operator([over.inc.default]), and

1.2 - not have default arguments ([dcl.fct.default]).

When adding the clause to define defaulted postfix operations, we note that existing wording in [over.inc] saves repetition by describing decrement operators as analogous to their increment counterparts. While we of course must define the behaviour of a defaulted decrement operator, we hope to use the same principle to avoid other repetition.

Add clause **§12.4.7.1 [over.inc.default]** as follows:

¹ A non-template postfix increment operator function may have an *explicitly defaulted definition* ([`dcl.fct.def.default`]). Such a function operating on some class type `C` shall:

1.1 - be a non-static member of `C` or a non-member function, and

1.2 - be defined as defaulted in `C` or in a context where `C` is complete, and

1.3 - have two parameters of (possibly different) type "reference to `C`" and `int` respectively, where the implicit object parameter (if any) is considered to be the first parameter, and

1.4 - have the return type `C`.

If type `C` does not have a declared copy constructor ([`class.copy.ctor`]) or prefix increment operator which is accessible from a context equivalent to the *function-body* of a defaulted postfix increment operator function, it is defined as deleted. A definition of a postfix increment operator function as defaulted that appears in a class shall be the first declaration of that function.

[Example 1:

```
struct S;
S operator++(S&, int) = default;           //error: S is not complete

struct S{
    S(const S&) = default;
    S& operator++(){ return *this; }
};
S operator++(S, int) = default;           //error: Incorrect parameter type

struct T{
    T operator++(int) = default;           //ok: Defined as deleted
};
```

]

² A non-template postfix decrement operator function also may have an *explicitly-defaulted definition*, and is handled analogously to the postfix increment operator function.

³ The behaviour of a defaulted postfix increment operator function operating on instance `c` of class type `C` shall be equivalent to:

```
C tmp = C{c};
++c;
return tmp;
```

The behaviour of a defaulted postfix decrement operator function operating on instance `c` of class type `C` shall be equivalent to:

```
C tmp = C{c};
--c;
return tmp;
```

§ 9.1. Wording relative to P2952

If [P2952] were to be accepted, we would additionally modify §12.4.7.1 [over.inc.default] as follows:

⁴ If the declared return type of a defaulted postfix increment operator or defaulted postfix decrement operator contains a placeholder type, its return type is deduced as if from `return r`, where `r` is an lvalue reference to an object of the type on which the operator is invoked.

§ References

§ Informative References

[P1046]

David Stone. *Automatically Generate More Operators*. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p1046r2.html>

[P1046-EWG]

EWG. *EWG comments from the Belfast meeting*. URL: <https://wiki.edg.com/bin/view/Wg21belfast/P1046-EWG>

[P1152]

JF Bastien. *Deprecating volatile*. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1152r4.html>

[P2727]

Zach Laine. *std::iterator interface*. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p2727r4.html>

[P2952]

Arthur O'Dwyer, Matthew Taylor. *auto& operator=(X&&) = default*. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p2952r2.html>

[P3039]

David Stone. *Automatically Generate operator->*. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p3039r0.html>

[P3294]

Andrei Alexandrescu; Barry Revzin; Daveed Vandevoorde. *Code Injection with Token Sequences*. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3294r2.html>

[P3662]

Jeremy Rifkin. *Improve Increment and Decrement Operator Syntax*. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3662r0.html>